Code Explanation

Data Writer Module Explanation

Overview

The DataWriter class is a Python module that handles writing output data to target systems, including flat files and staging tables. It utilizes PySpark DataFrames and configuration settings to determine the output format and location.

Step 1: Initializing the DataWriter Class

The DataWriter class is initialized with a Config object, which provides the necessary configuration settings for the data writing process.

```
def __init__(self):
    self.config = Config()
```

Step 2: Writing Flat File Output

The `write_flat_file_output` method writes a PySpark DataFrame to a flat file in the specified output format (either CSV or Parquet).

```
def write_flat_file_output(self, df: DataFrame) -> None:
    output_format = self.config.OUTPUT_FORMAT
    output_path = self.config.FLAT_FILE_OUTPUT_PATH

    if output_format == "csv":
        df.write.mode("overwrite").option("header", "true").option(
            "encoding", "UTF-8"
        ).csv(output_path)
    else:
        df.write.mode("overwrite").parquet(output_path)

    print(f"Flat file output written to: {output_path}")
```

Step 3: Updating the Staging Table

The `update_staging_table` method updates the staging table with process stamps. It creates a temporary view for the update logic and writes the updates to a separate location (in this case, a Parquet file).

```
def update_staging_table(self, df: DataFrame) -> None:
    # Create temporary view for update logic
    df.createOrReplaceTempView("staging_updates")

    # For demonstration, write to separate location
    update_path = f"{self.config.OUTPUT_BASE_PATH}/staging_updates"
    df.write.mode("overwrite").parquet(update_path)

    print(f"Staging updates written to: {update_path}")
    print(f"Records updated: {df.count()}")
```

In a production environment, this method would use JDBC with update mode to update the staging table. However, in this example, it simulates the update by writing to a separate location or using a merge/upsert pattern with Delta Lake.

The code is well-structured, readable, and follows proper indentation and markdown formatting for clarity. The use of PySpark DataFrames and configuration settings makes the code flexible and adaptable to different output formats and locations.